Dateiformate

JSON, lesbares TXT und C-Header

Inhaltsverzeichnis

Genauigkeit 1
JSON 1
TXT 2
C-Header 2

Ein trainiertes Netz besteht nur aus Zahlen: den Gewichten und Biases aller Neuronen. Die Angabe verlangt drei Formate dafür, jedes für einen anderen Zweck. Python und C# schreiben und lesen dieselben Dateien.

Format	Zweck	Methode
`.json`	Netz speichern und wieder laden	`dump_json(path)`, `NeuralNetwork.load(path)`
`.txt`	für Menschen lesbar; in Teil 4 lädt ein C-Programm das Netz daraus	`dump_txt(path)`, `NeuralNetwork.load_txt(path`
`.h`	C-Header mit allen Werten als Konstanten, für Teil 5 und den ESP32	`dump_initfile(path, prefix)`

Alle Beispiele unten sind das trainierte XOR-Netz aus den Ergebnissen.

Genauigkeit

Zahlen werden so geschrieben, dass beim Einlesen **exakt dieselbe** Gleitkommazahl herauskommt (kürzeste eindeutige Darstellung, in Python `repr`). Ein neu geladenes Netz sagt deshalb bitgleich dasselbe voraus wie das Original — die Tests prüfen das mit der Differenz 0.

In C# muss beim Schreiben und Lesen `CultureInfo.InvariantCulture` verwendet werden. Mit deutschen Ländereinstellungen entstünde sonst `5,9005` statt `5.9005`, und kein anderes Programm könnte die Datei lesen.

JSON

Die Struktur bildet das Netz direkt ab: eine Liste von Layern, jeder mit einer Liste von Neuronen. `format` und `version` erlauben es, falsche oder veraltete Dateien sofort zu erkennen.

```
{
  "format": "insy-nn",
  "version": 1,
```

```
  "activation": "sigmoid",
  "input_size": 2,
  "layers": [
    {
      "neurons": [
            { "weights": [5.900521966319981, -5.914256815253622], "bias":
-3.357581665610364 },
            { "weights": [-5.421024708486351, 5.135707632445938], "bias":
-2.859692553869736 }
      ]
    },
    {
      "neurons": [
             { "weights": [8.158894259030141, 8.307855908650177], "bias":
-4.0624992872092545 }
      ]
    }
  ]
}
```

Beim Laden wird geprüft, ob jedes Neuron so viele Gewichte hat wie der vorherige Layer Neuronen (bzw. das Netz Inputs).

TXT

Vor jedem Wert steht ein Schlüsselwort. So liest sich die Datei wie eine Auflistung, und ein C-Programm kann sie mit `fscanf` Wort für Wort einlesen, ohne einen JSON-Parser zu brauchen.

```
insy-nn 1
activation sigmoid
input_size 2
layers 2

layer 1 neurons 2 inputs 2
neuron 1 bias -3.357581665610364 weights 5.900521966319981 -5.914256815253622
neuron 2 bias -2.859692553869736 weights -5.421024708486351 5.135707632445938

layer 2 neurons 1 inputs 2
neuron 1 bias -4.0624992872092545 weights 8.158894259030141 8.307855908650177
```

C-Header

Der Header enthält Größen und alle Werte als Konstanten. Das Präfix (hier `XOR`) steht vor jedem Namen, damit mehrere Netze in einem Programm Platz haben. `WEIGHTS[n][m]` heißt: Neuron `n` dieses Layers, Gewicht für Input `m`.

```c
// Generated by neural_network (insy-nn v1). Do not edit.
// Activation: sigmoid
#ifndef XOR_NET_H
#define XOR_NET_H

#define XOR_INPUT_SIZE 2
#define XOR_LAYER_COUNT 2

static const int XOR_LAYER_SIZES[XOR_LAYER_COUNT] = {2, 1};

// Layer 1: 2 neurons x 2 inputs
static const double XOR_L1_WEIGHTS[2][2] = {
    {5.900521966319981, -5.914256815253622},
    {-5.421024708486351, 5.135707632445938},
};
static    const    double    XOR_L1_BIASES[2]    =    {-3.357581665610364,
-2.859692553869736};

// Layer 2: 1 neurons x 2 inputs
static const double XOR_L2_WEIGHTS[1][2] = {
    {8.158894259030141, 8.307855908650177},
};
static const double XOR_L2_BIASES[1] = {-4.0624992872092545};

#endif // XOR_NET_H
```

Die Angabe wünscht im Header auch eine Funktion, die das Netz initialisiert. Sie braucht die C-Strukturen (`Neuron`, `Layer`, `Network`) aus Teil 4 und kommt deshalb mit Teil 4 dazu.